

Project 2 Report: Employee Attrition

Stacy Bowler
Utah State University
stacy.bowler@sdl.usu.edu

Derrik Miller
Utah State University
derrik.miller@sdl.usu.edu

2026-08-07

Abstract This project develops a blank.

Introduction

The objective of this project is to develop a predictive model capable of determining which employees are most likely to leave. This can allow the business to focus their retention efforts on high-risk employees.

The business decision addressed by this analysis is:

Which employees are likely to leave?

The resulting model can be used to identify employees that appear likely to leave.

Business Objective

The primary objective is to accurately determine which employees are likely to quit before they do.

A practical utility function for this problem is:

$$U = \begin{cases} 0.60 \times 40000 - 8000 = 16000 & \text{if a retention intervention is provided and prevents attrition} \\ -8000 & \text{if a retention intervention is provided unnecessarily} \\ -40000 & \text{if no intervention is provided and the employee leaves} \\ 0 & \text{if no intervention is provided and the employee stays} \end{cases}$$

assumptions:

- Cost of employee turnover: \$40,000
- Cost of retention intervention: \$8,000
- Retention intervention success rate: 60%

Plan

The analysis follows the regression modeling workflow:

1. Define the business problem.
2. Acquire and clean employee data.
3. Explore relationships between candidate predictors and attrition.
4. Build an interpretable regression model.
5. Evaluate assumptions and model fit.
6. Generate predictions and business recommendations.

The ideal dataset would contain:

- Age
- Attrition
- Salary
- Years at the company
- Overtime
- Job role
- Job satisfaction score

Due to the data being fake generated data, it was more than ideal. It included everything we thought of and much more.

Build

Data Source

The dataset was obtained from Kaggle at <https://www.kaggle.com/datasets/pavansubhasht/ibm-hr-analytics-attrition-dataset> and contains a fictional dataset created by IBM data scientists.

Variables initially available included 35 different point including notably:

- Age
- Attrition
- Department
- Gender
- Job Role
- Job Satisfaction
- Martial Status
- Monthly Income
- Number of Companies Worked
- Overtime
- Years Since Last Promotion
- Year at Company
- Years in Current Role

Data Cleaning

The following preprocessing steps were performed:

- Converted Attrition to a binary outcome where Yes = 1

```

import os
import numpy as np
import pandas as pd
from patsy import dmatrices
import polars as pl
import seaborn.objects as so
import seaborn as sns
import matplotlib.pyplot as plt
import statsmodels.api as sm
import statsmodels.formula.api as smf
from sklearn.model_selection import train_test_split
from statsmodels.stats.outliers_influence import variance_inflation_factor

pl.Config.set_tbl_rows(-1)
pl.Config.set_tbl_cols(-1)
pl.Config.set_fmt_str_lengths(100)

employee_data = (pl.read_csv(os.path.join('data', 'HR-Employee-Attrition.csv'))
    # Make the outcome binary where 'Yes' = 1
    .with_columns(
        pl.when(pl.col('Attrition') == 'Yes').then(1)
        .when(pl.col('Attrition') == 'No').then(0)
        .alias('Left')
    )
    .select(pl.exclude(['EmployeeNumber', 'DailyRate', 'MonthlyIncome',
        'EmployeeCount', 'Over18', 'PerformanceRating']))
)

# Print unique values and their counts
# for col in employee_data.columns:
#     print(f"\n{'='*60}")
#     print(f"Column: {col}")
#     print(f"{'='*60}")

#     print(
#         employee_data
#         .group_by(col)
#         .len()
#         .sort("len", descending=True)
#     )

predictors = [
    "Age",

```

```

"C(BusinessTravel)",
# "C(Department)", # job role perfectly maps to certain departments except
manager
"DistanceFromHome",
"C(EducationField)", # This seems to be redundant.
"C(Education)",
"EnvironmentSatisfaction",
"C(Gender)",
"HourlyRate", # Hourly rate seems to group employees together better than other
similar rate columns that tend to only match one or few employees.
"JobInvolvement",
"C(JobLevel)",
"C(JobRole)",
"JobSatisfaction",
"C(MaritalStatus)",
"MonthlyRate", # may be more linear and likely redundant with hourly rate
"NumCompaniesWorked",
"C(OverTime)",
"PercentSalaryHike",
"RelationshipSatisfaction",
"TotalWorkingYears", # may correlate strongly with Age
"TrainingTimesLastYear",
"WorkLifeBalance",
"YearsAtCompany",
"YearsInCurrentRole",
"YearsSinceLastPromotion",
"YearsWithCurrManager"
]

```

Explore

Exploratory Data Analysis

Initial visualizations focused on the relationships between attrition and major numerical predictors.

Overtime vs Attrition

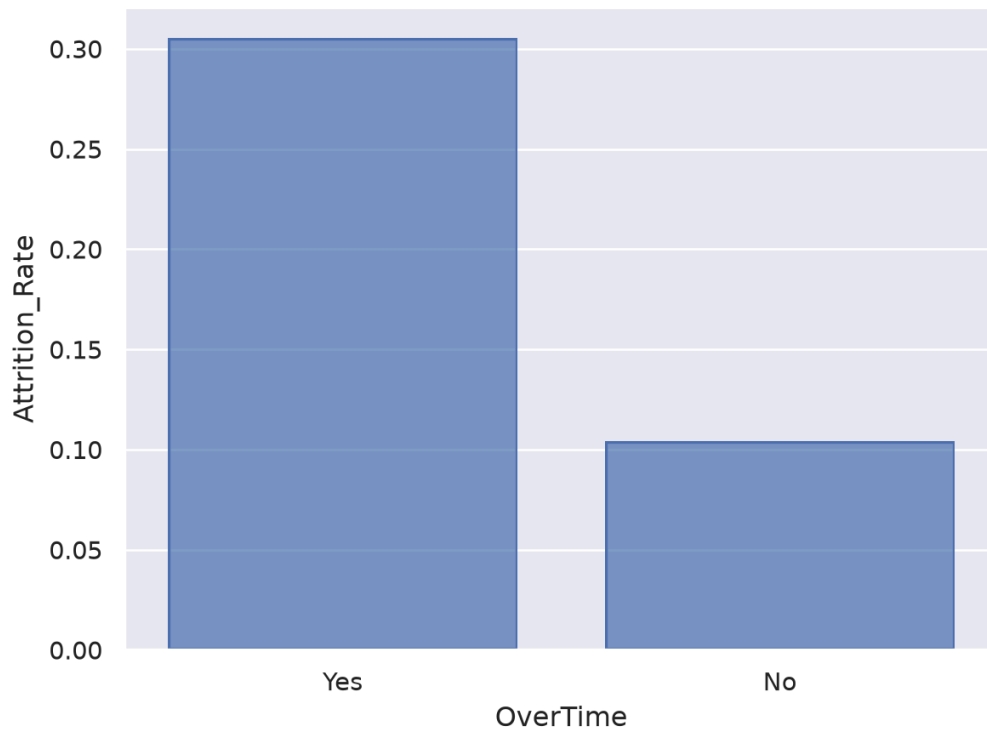
```

overtime_rate = (
    employee_data
    .group_by("OverTime")
    .agg(Attrition_Rate=pl.col("Left").mean())
)

so.Plot(overtime_rate.to_pandas(),

```

```
x="OverTime",
y="Attrition_Rate") \
.add(so.Bar())
```



This looks like employees that work overtime are more likely to leave.

Satisfaction vs Attrition

```
(
    so.Plot(employee_data.to_pandas(),
            x="JobSatisfaction",
            y="Left")
    .add(so.Dot(alpha=.20))
    .add(so.Line(), so.PolyFit(order=1))
).show()

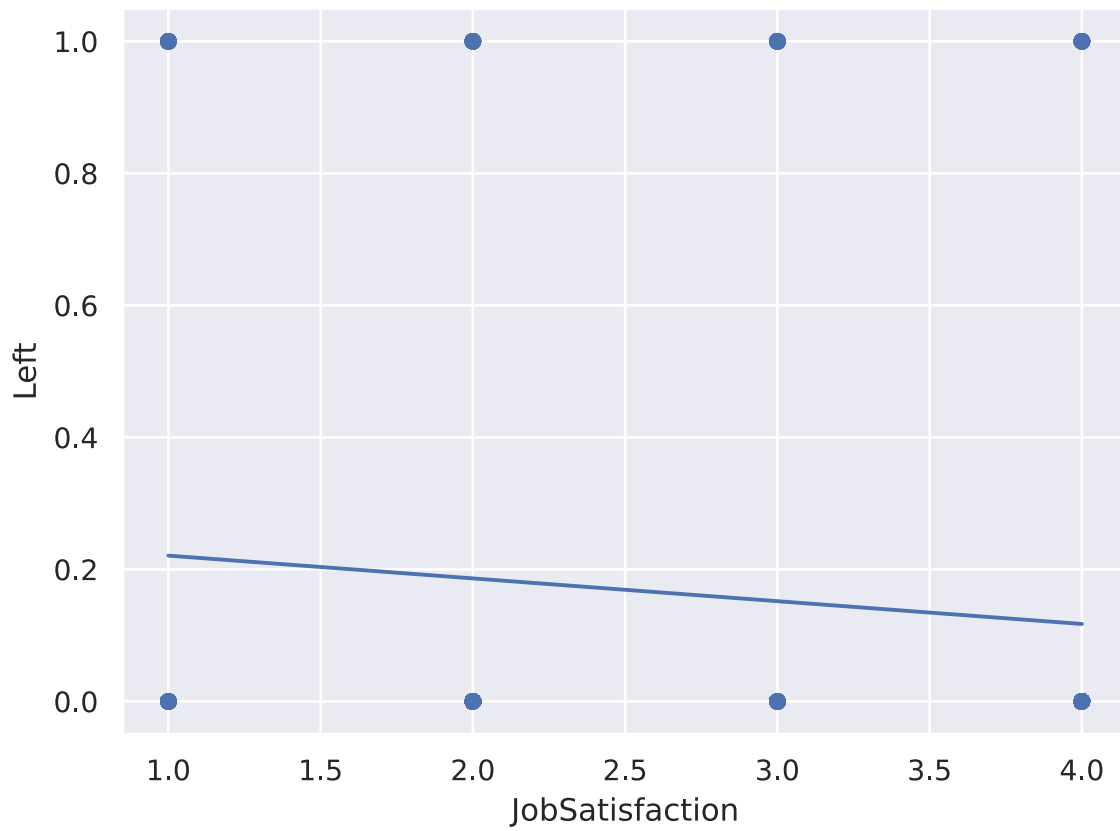
(
    so.Plot(employee_data.to_pandas(),
            x="EnvironmentSatisfaction",
            y="Left")
    .add(so.Dot(alpha=.20))
    .add(so.Line(), so.PolyFit(order=1))
```

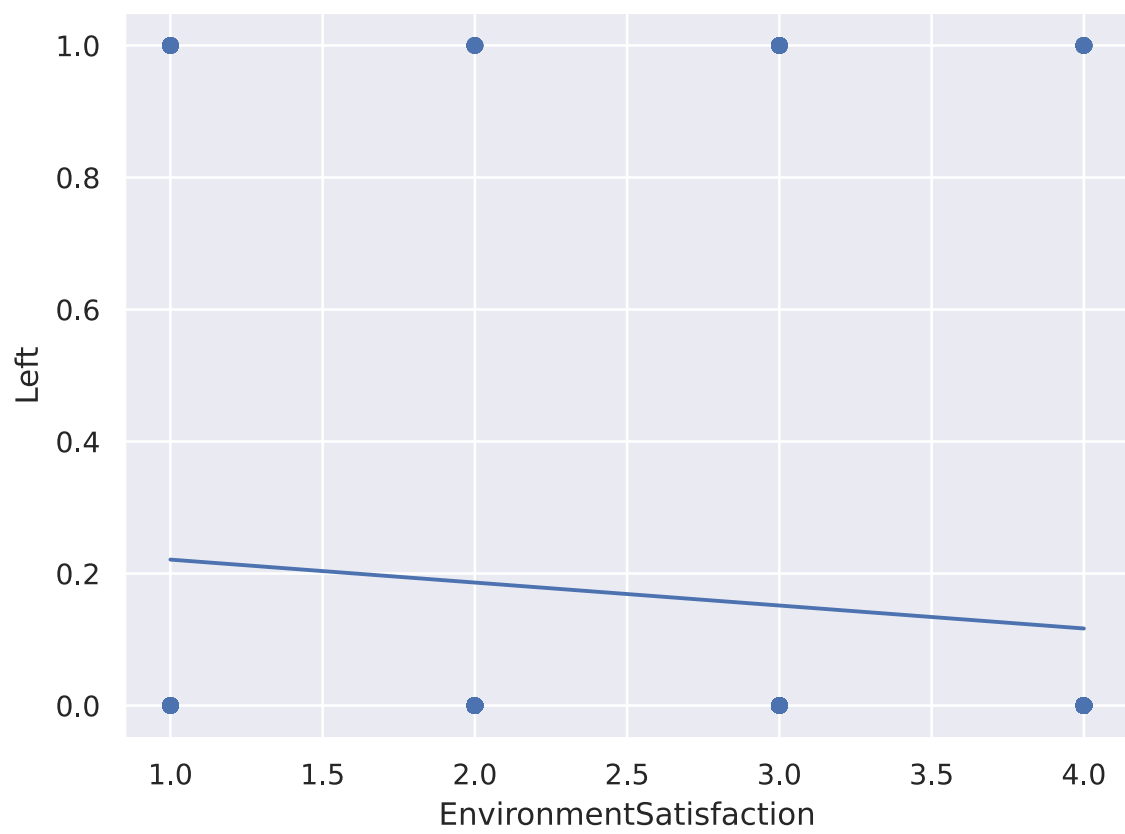
```

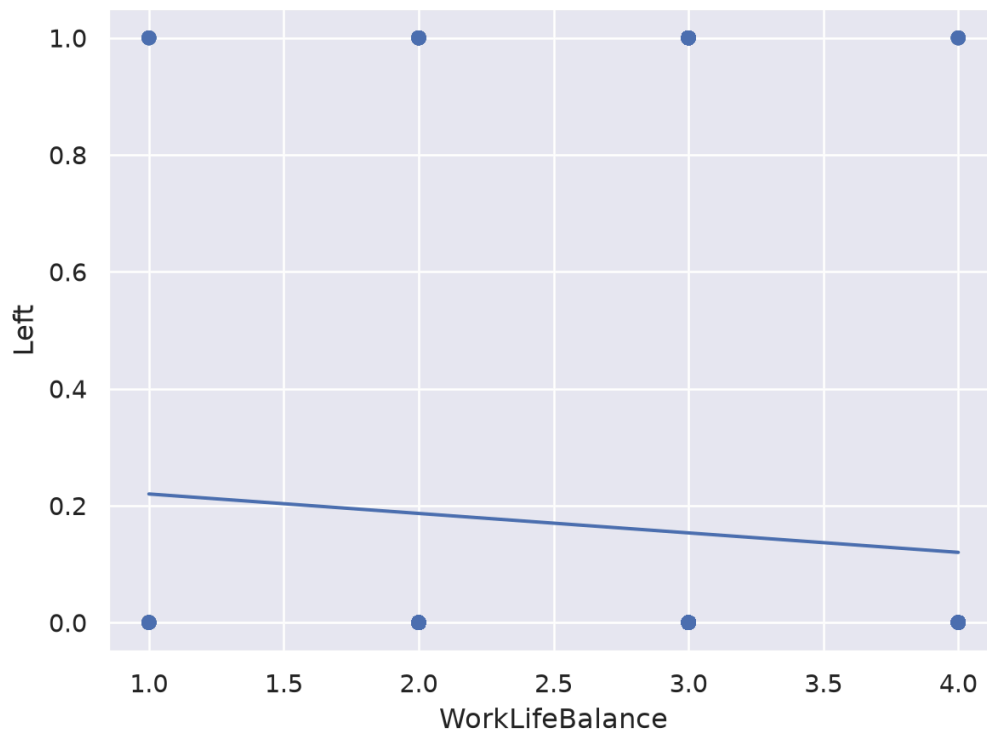
).show()

(
    so.Plot(employee_data.to_pandas(),
            x="WorkLifeBalance",
            y="Left")
    .add(so.Dot(alpha=.20))
    .add(so.Line(), so.PolyFit(order=1))
)

```



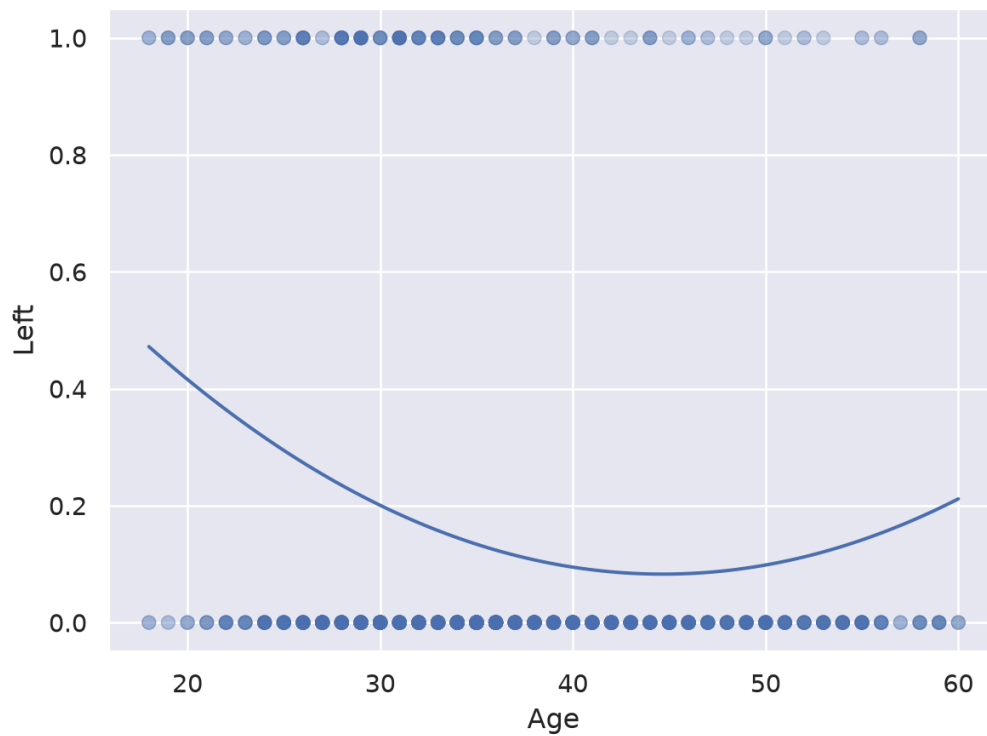




These graphs show what we would expect, that as employees report being more satisfied or having better work/life balance, they are less likely to leave.

Age vs Attrition

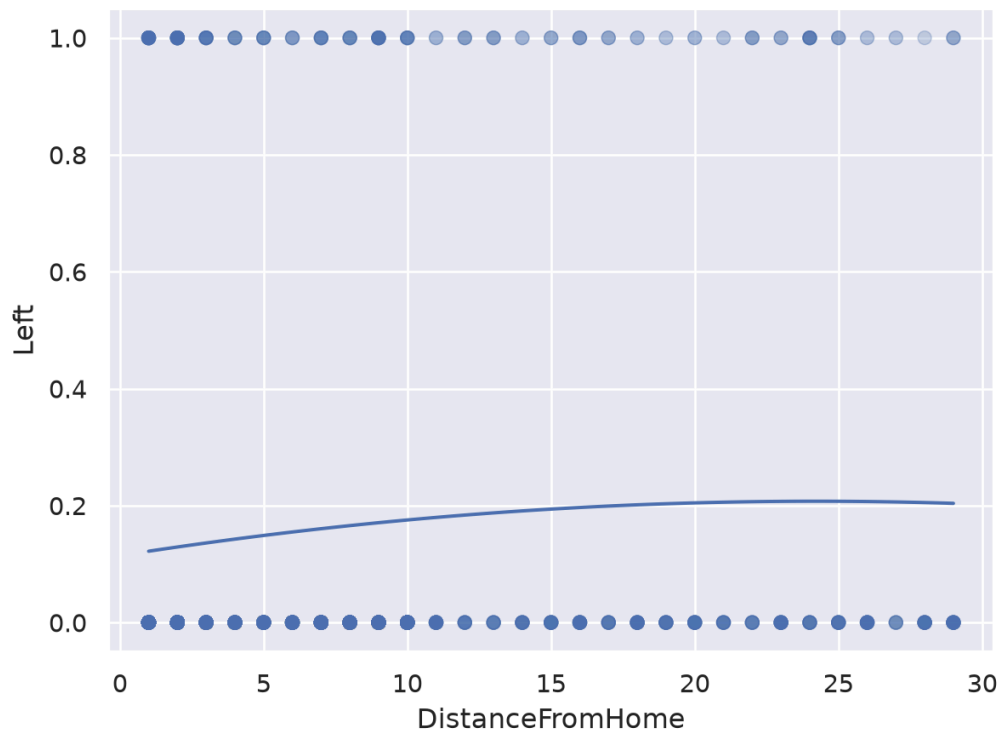
```
(  
    so.Plot(employee_data.to_pandas(),  
            x="Age",  
            y="Left")  
    .add(so.Dot(alpha=.15))  
    .add(so.Line(), so.PolyFit(order=2))  
)
```



We can see that younger employees are generally more likely to leave, but this starts to reverse around age 45. Retirement is likely a factor, but also employees are likely changing jobs for more senior roles.

Commute vs Attrition

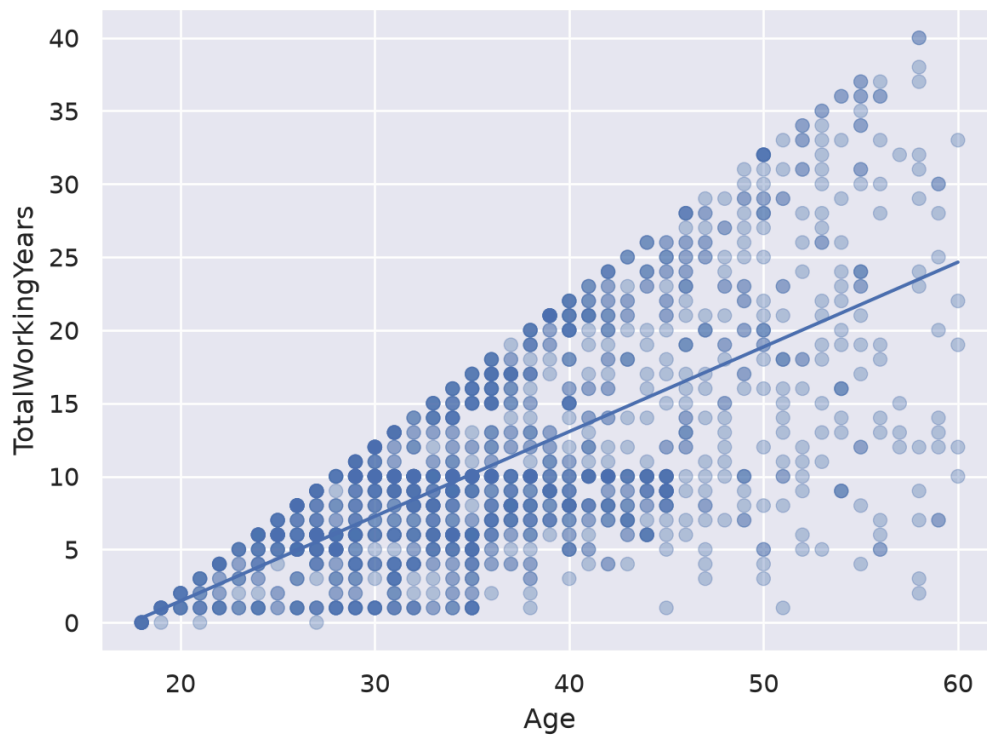
```
(  
    so.Plot(employee_data.to_pandas(),  
            x="DistanceFromHome",  
            y="Left")  
    .add(so.Dot(alpha=.15))  
    .add(so.Line(), so.PolyFit(order=2))  
)
```



There is a fairly weak correlation between how far someone lives from work and if they are likely to leave. It is interesting that the trend appears to level off so it seems like it's only a factor within a certain range.

Age vs Total Working Years

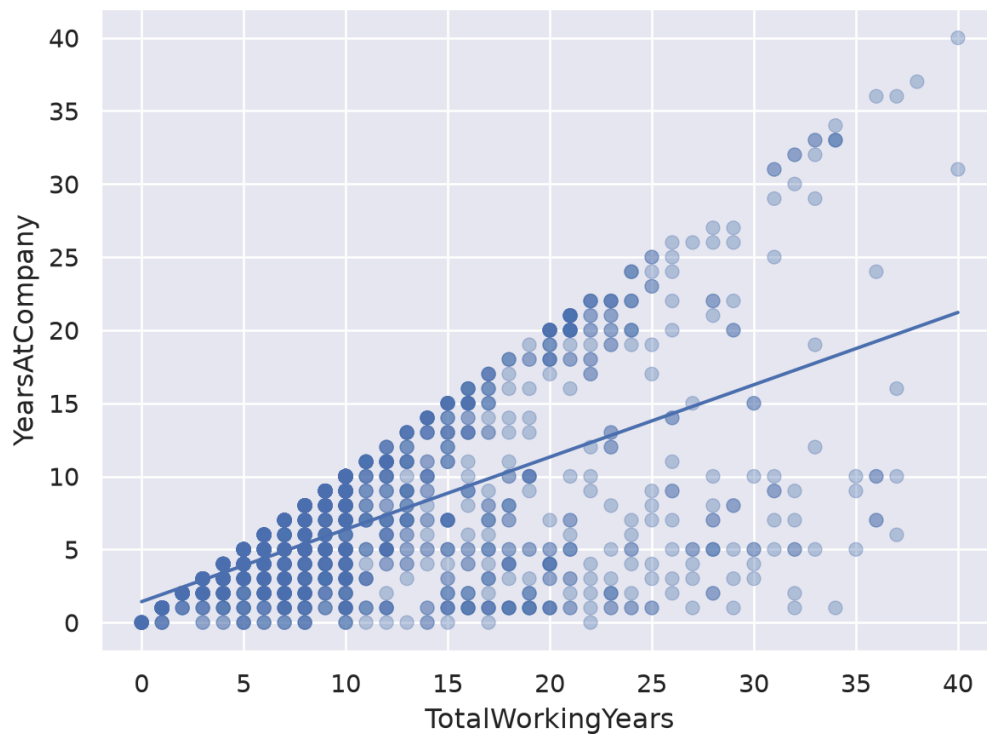
```
(
    so.Plot(employee_data.to_pandas(),
            x="Age",
            y="TotalWorkingYears")
    .add(so.Dot(alpha=.35))
    .add(so.Line(), so.PolyFit(order=1))
)
```



We thought there would be a strong relationship in the data that could cause issues between employee age and total working years. It looks like there is a relationship, but the correlation is not nearly as strong as we thought.

Years at Company vs Total Working Years

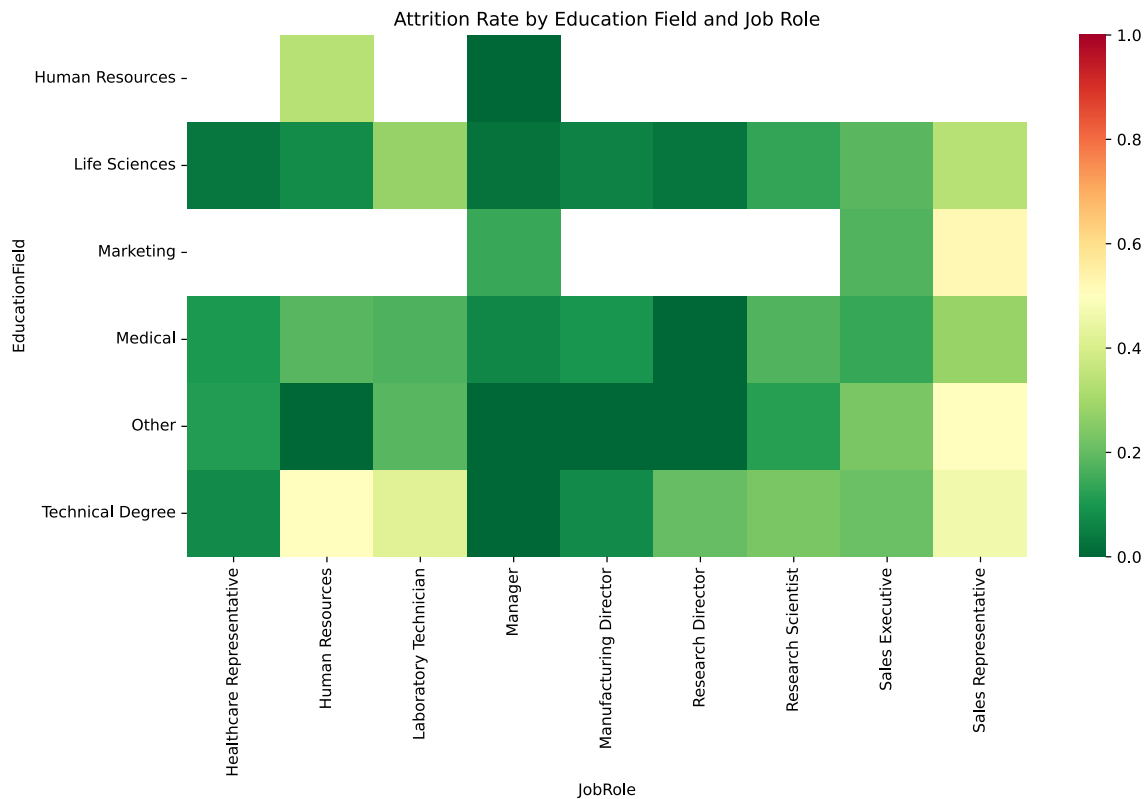
```
(  
    so.Plot(employee_data.to_pandas(),  
            x="TotalWorkingYears",  
            y="YearsAtCompany")  
    .add(so.Dot(alpha=.35))  
    .add(so.Line(), so.PolyFit(order=1))  
)
```



It looks like there is a relationship between years at the company and total working years, but not a very strong one.

Education Field vs Job Role

```
rate = (
    employee_data
    .group_by(["EducationField", "JobRole"])
    .agg(attrition_rate=pl.col("Left").mean())
    .to_pandas()
)
pivot_rate = rate.pivot(
    index="EducationField",
    columns="JobRole",
    values="attrition_rate"
)
plt.figure(figsize=(12,6))
sns.heatmap(pivot_rate, cmap="RdYlGn_r", vmin=0, vmax=1)
plt.title("Attrition Rate by Education Field and Job Role")
plt.show()
```

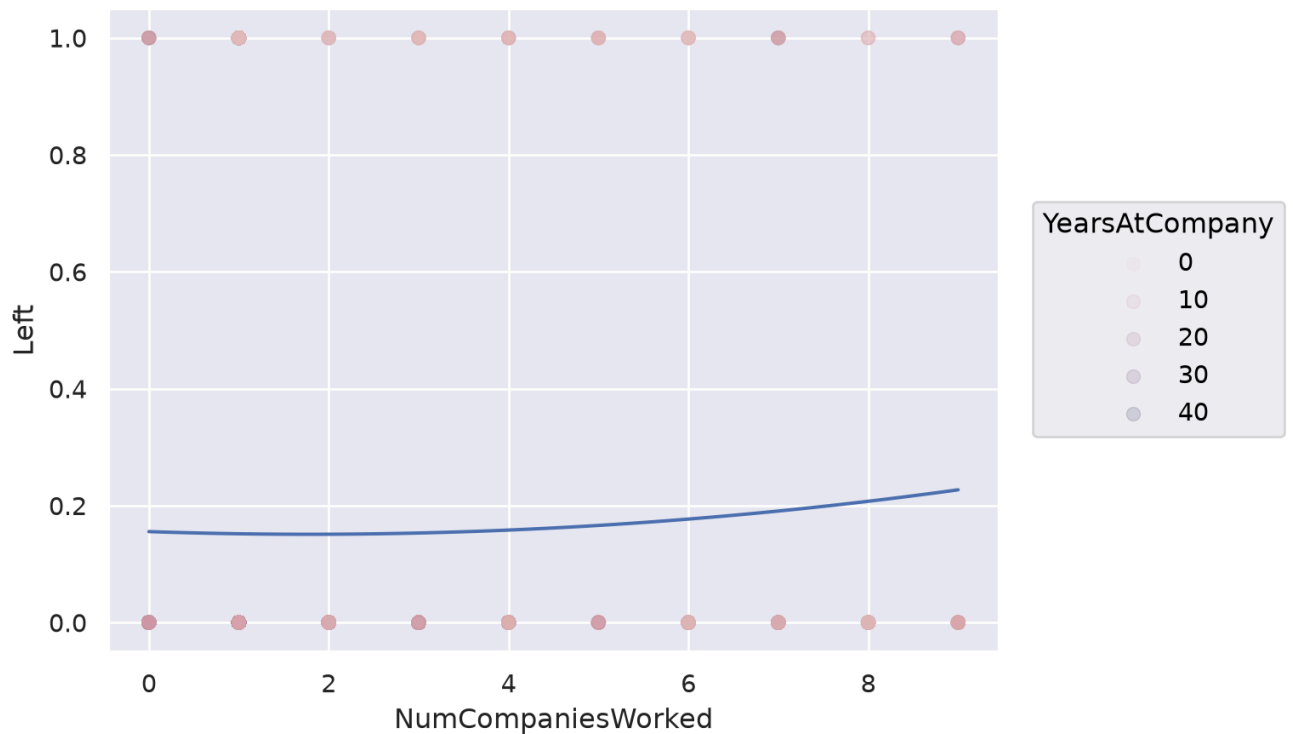


The heatmap reveals a strong structural relationship between education field and job role, indicating that employees are not randomly distributed across roles. Instead, specific education backgrounds concentrate within particular job families. This dependency explains multicollinearity we later observed in the initial logistic regression model.

Attrition varies more meaningfully across job roles than across education fields, suggesting that occupational role is a stronger predictor of turnover than educational background.

Number of Companies and Attrition

```
(
  so.Plot(employee_data.to_pandas(),
    x="NumCompaniesWorked",
    y="Left")
  .add(so.Dot(alpha=.15), color="YearsAtCompany")
  .add(so.Line(), so.PolyFit(order=2))
)
```



This is an interesting graph because it shows that employees that have worked at more companies are more likely to leave. This could be explained by the fact that employees that have worked at more companies are more likely to be looking for new opportunities. It also shows that employees that have been at the company for a long time are less likely to leave, which makes sense.

Key Findings

- Found this.
- Found that.

Reconcile

Sampling Strategy

A stratified sampling approach was used to evenly split test and training data with even attrition and non-attrition employees.

Assumptions

Model Refinements

Several iterations of the model were evaluated.

Adjustments included:

- abcd
- GDs.

- Evaluating Variance Inflation Factors (VIFs).

```
pb_train = employee_data.to_pandas()

train_df, test_df = train_test_split(
    pb_train,
    test_size=0.30,
    random_state=42,
    stratify=pb_train["Left"]
)

formula = "Left ~ " + " + ".join(predictors)

fit_01 = smf.logit(
    formula,
    data=train_df
).fit()

print(fit_01.summary())

# Create the design matrix used by the model
y, X = dmatrices(formula, data=pb_train, return_type="dataframe")

# Calculate VIF
vif = pd.DataFrame({
    "Variable": X.columns,
    "VIF": [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
})

print(vif.sort_values("VIF", ascending=False))
```

```
Optimization terminated successfully.
    Current function value: 0.271364
    Iterations 9
```

Logit Regression Results			
=====			
Dep. Variable:	Left	No. Observations:	1029
Model:	Logit	Df Residuals:	984
Method:	MLE	Df Model:	44
Date:	Fri, 17 Jul 2026	Pseudo R-squ.:	0.3859
Time:	21:38:00	Log-Likelihood:	-279.23
converged:	True	LL-Null:	-454.67
Covariance Type:	nonrobust	LLR p-value:	1.916e-49

z		coef	std err	z	P>
[0.025 0.975]					
Intercept		4.1633	1.769	2.354	0.019
0.697	7.630				
C(BusinessTravel)[T.Travel_Frequently]		2.0156	0.514	3.919	0.000
1.008	3.024				
C(BusinessTravel)[T.Travel_Rarely]		1.0539	0.467	2.254	0.024
0.138	1.970				
C(EducationField)[T.Life Sciences]		-1.7028	0.933	-1.825	0.068
-3.532	0.126				
C(EducationField)[T.Marketing]		-1.4330	0.980	-1.463	0.144
-3.353	0.487				
C(EducationField)[T.Medical]		-1.8102	0.936	-1.933	0.053
-3.646	0.025				
C(EducationField)[T.Other]		-2.3162	1.057	-2.191	0.028
-4.388	-0.245				
C(EducationField)[T.Technical Degree]		-1.2318	0.952	-1.294	0.196
-3.098	0.634				
C(Education)[T.2]		0.0728	0.406	0.179	0.858
-0.722	0.868				
C(Education)[T.3]		0.1837	0.347	0.529	0.597
-0.497	0.865				
C(Education)[T.4]		0.0287	0.383	0.075	0.940
-0.721	0.778				
C(Education)[T.5]		0.1043	0.750	0.139	0.889
-1.366	1.575				
C(Gender)[T.Male]		0.3303	0.234	1.411	0.158
-0.128	0.789				
C(JobLevel)[T.2]		-1.4576	0.526	-2.770	0.006
-2.489	-0.426				
C(JobLevel)[T.3]		-0.0543	0.680	-0.080	0.936
-1.387	1.278				
C(JobLevel)[T.4]		-0.6839	1.083	-0.632	0.528
-2.806	1.438				
C(JobLevel)[T.5]		1.0270	1.409	0.729	0.466
-1.734	3.788				
C(JobRole)[T.Human Resources]		0.9110	0.942	0.968	0.333
-0.934	2.756				
C(JobRole)[T.Laboratory Technician]		1.2668	0.722	1.754	0.079
-0.149	2.682				
C(JobRole)[T.Manager]		-0.1943	0.998	-0.195	0.846

-2.151	1.762				
C(JobRole)[T.Manufacturing Director]		0.5552	0.721	0.770	0.441
-0.858	1.968				
C(JobRole)[T.Research Director]		-2.0340	1.390	-1.464	0.143
-4.758	0.690				
C(JobRole)[T.Research Scientist]		-0.1597	0.762	-0.210	0.834
-1.654	1.334				
C(JobRole)[T.Sales Executive]		1.6846	0.606	2.782	0.005
0.498	2.872				
C(JobRole)[T.Sales Representative]		1.7233	0.815	2.115	0.034
0.126	3.321				
C(MaritalStatus)[T.Married]		0.4107	0.321	1.279	0.201
-0.219	1.040				
C(MaritalStatus)[T.Single]		1.6026	0.336	4.769	0.000
0.944	2.261				
C(OverTime)[T.Yes]		2.0727	0.251	8.273	0.000
1.582	2.564				
Age		-0.0420	0.018	-2.390	0.017
-0.076	-0.008				
DistanceFromHome		0.0502	0.014	3.714	0.000
0.024	0.077				
EnvironmentSatisfaction		-0.4987	0.106	-4.693	0.000
-0.707	-0.290				
HourlyRate		-0.0025	0.006	-0.446	0.656
-0.013	0.008				
JobInvolvement		-0.5621	0.150	-3.747	0.000
-0.856	-0.268				
JobSatisfaction		-0.3862	0.101	-3.838	0.000
-0.583	-0.189				
MonthlyRate		2.351e-05	1.56e-05	1.504	0.133
-7.14e-06	5.42e-05				
NumCompaniesWorked		0.2196	0.049	4.499	0.000
0.124	0.315				
PercentSalaryHike		-0.0171	0.031	-0.547	0.584
-0.078	0.044				
RelationshipSatisfaction		-0.3647	0.102	-3.560	0.000
-0.566	-0.164				
TotalWorkingYears		-0.0687	0.036	-1.903	0.057
-0.140	0.002				
TrainingTimesLastYear		-0.1471	0.091	-1.618	0.106
-0.325	0.031				
WorkLifeBalance		-0.4242	0.158	-2.690	0.007
-0.733	-0.115				

YearsAtCompany	0.0197	0.052	0.380	0.704
-0.082 0.121				
YearsInCurrentRole	-0.1245	0.060	-2.060	0.039
-0.243 -0.006				
YearsSinceLastPromotion	0.1914	0.053	3.594	0.000
0.087 0.296				
YearsWithCurrManager	-0.0920	0.063	-1.466	0.143
-0.215 0.031				

	Variable	VIF
0	Intercept	230.084646
3	C(EducationField)[T.Life Sciences]	20.401900
5	C(EducationField)[T.Medical]	18.316684
4	C(EducationField)[T.Marketing]	9.216272
7	C(EducationField)[T.Technical Degree]	7.600715
6	C(EducationField)[T.Other]	5.234651
38	TotalWorkingYears	5.098665
41	YearsAtCompany	4.781034
15	C(JobLevel)[T.4]	4.736300
16	C(JobLevel)[T.5]	4.292659
22	C(JobRole)[T.Research Scientist]	4.251536
14	C(JobLevel)[T.3]	3.908152
18	C(JobRole)[T.Laboratory Technician]	3.898645
13	C(JobLevel)[T.2]	3.749742
23	C(JobRole)[T.Sales Executive]	3.172551
19	C(JobRole)[T.Manager]	2.999902
44	YearsWithCurrManager	2.902772
42	YearsInCurrentRole	2.850916
9	C(Education)[T.3]	2.799689
10	C(Education)[T.4]	2.669351
2	C(BusinessTravel)[T.Travel_Rarely]	2.384178
1	C(BusinessTravel)[T.Travel_Frequently]	2.371898
24	C(JobRole)[T.Sales Representative]	2.317760
8	C(Education)[T.2]	2.261598
21	C(JobRole)[T.Research Director]	2.231553
28	Age	2.104707
17	C(JobRole)[T.Human Resources]	2.049146
20	C(JobRole)[T.Manufacturing Director]	1.940330
43	YearsSinceLastPromotion	1.719382
26	C(MaritalStatus)[T.Single]	1.711571
25	C(MaritalStatus)[T.Married]	1.690324
11	C(Education)[T.5]	1.303642
35	NumCompaniesWorked	1.291810

39	TrainingTimesLastYear	1.040054
27	C(OverTime)[T.Yes]	1.036543
12	C(Gender)[T.Male]	1.033266
30	EnvironmentSatisfaction	1.033092
32	JobInvolvement	1.029939
40	WorkLifeBalance	1.028596
29	DistanceFromHome	1.028264
37	RelationshipSatisfaction	1.028146
34	MonthlyRate	1.027966
31	HourlyRate	1.025140
33	JobSatisfaction	1.024686
36	PercentSalaryHike	1.024530

Given the high P values, we should consider removing these from our model

Variable	p-value
Education 2–5	0.60–0.94
HourlyRate	0.656
PercentSalaryHike	0.584
YearsAtCompany	0.704

Then these next | Variable | p-value | | ----- | ---: | | JobLevel 3 | 0.936 | | JobLevel 4 | 0.528 | | JobLevel 5 | 0.466 | | JobRole Manager | 0.846 | | JobRole Research Scientist | 0.834 | | Manufacturing Director | 0.441 |

Fit

Final Model

The final model included:

- Param 1
- Param 2

```
# fit_log.summary()
```

The fitted model produced:

- $R^2 = 0.818$
- Adjusted $R^2 = 0.812$
- F-statistic = 141.6
- $p < 0.001$

indicating strong overall explanatory power.

Important Continuous Predictors

Variable	Coefficient
OverTime	2.07
Business Travel	2.02/1.05
Marital Status	1.60

OverTime

Employees working overtime have roughly 8 times the odds of attrition compared with employees who do not, holding the other variables constant:

$$e^{2.07} \approx 7.95$$

Business Travel

Employees traveling frequently have roughly 7.5 times the odds of attrition compared to employees who do not travel, holding the other variables constant:

$$e^{2.02} \approx 7.5$$

Employees traveling rarely also have roughly 2.9 times the odds of attrition compared to employees who do not travel, holding the other variables constant:

$$e^{1.05} \approx 2.9$$

Marital Status

Single employees have roughly 5 times the odds of attrition compared to divorced employees:

$$e^{1.60} \approx 5$$

Evaluate

Residual Analysis

Residual diagnostics indicated:

- No severe violations of linearity.
- Acceptable residual symmetry.
- Some heavy tails remain.
- No extreme multicollinearity concerns.

The largest VIF values were associated with popular truck categories but remained below commonly cited thresholds requiring corrective action.

Limitations

Several limitations should be noted:

- No MSRP information was available.
- Trim level information was not incorporated.
- Accident history was unavailable.

- Maintenance records were unavailable.
- Auction data may not represent all vehicle markets.

These limitations likely explain a portion of the unexplained variance.

Predict

Penalized Regression

```
import numpy as np
import pandas as pd
from patsy import dmatrices
from sklearn.linear_model import LogisticRegressionCV
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

# 1. Prepare Design Matrices using the formula
y, X = dmatrices(formula, data=pb_train, return_type="dataframe")
# Flatten y to a 1D array for scikit-learn
y = y.values.ravel()

# Split into train/test using the same indices or exact split structure
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y
)

# 2. Scale features (Crucial for regularized models)
# Note: Intercept column from patsy should not be scaled, or can be dropped
# since LogisticRegression adds its own intercept.
X_train_clean = X_train.drop(columns=['Intercept'], errors='ignore')
X_test_clean = X_test.drop(columns=['Intercept'], errors='ignore')

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_clean)
X_test_scaled = scaler.transform(X_test_clean)

# 3. Fit Models with Cross-Validation for Hyperparameter Tuning
cv_folds = 5

# Ridge Regression (L2)
ridge_cv = LogisticRegressionCV(l1_ratios=(0,), Cs=10, cv=cv_folds,
                                scoring='neg_log_loss', use_legacy_attributes=True, random_state=42,
                                max_iter=1000)
ridge_cv.fit(X_train_scaled, y_train)
```

```

# LASSO Regression (L1)
lasso_cv = LogisticRegressionCV(l1_ratios=(1,), solver='saga',
Cs=10, cv=cv_folds, scoring='neg_log_loss', use_legacy_attributes=True,
random_state=42, max_iter=5000)
lasso_cv.fit(X_train_scaled, y_train)

# Elastic Net Regression
elastic_cv = LogisticRegressionCV(l1_ratios=[0.1, 0.5, 0.7, 0.9], solver='saga',
Cs=10, cv=cv_folds, scoring='neg_log_loss', use_legacy_attributes=True,
random_state=42, max_iter=5000)
elastic_cv.fit(X_train_scaled, y_train)

```

l1_ratios l1_ratios: array-like [0.1, 0.5, ...]
of shape (n_l1_ratios), de-
fault=None

Floats between 0 and 1 passed
as Elastic-Net mixing para-
meter (scaling between
L1 and L2 penalties). For
`l1\_ratio = 0` the penalty is an
L2 penalty. For
`l1\_ratio = 1` it is an L1
penalty. For `0 < l1\_ratio < 1`,
the penalty is a
combination of L1 and L2.
All the values of the given
array-like are tested by cross-
validation and the
one giving the best prediction
score is used.

.. warning::

Certain values of `l1\_ratios`,
i.e. some penalties, may not
work with some
solvers. See the parameter
`solver` below, to know the
compatibility between
the penalty and solver.

.. deprecated:: 1.8

``l1_ratios=None`` is deprecated in 1.8 and will raise an error in version 1.10. Default value will change from ``None`` to ``(0.0,)`` in version 1.10.

cv cv: int or cross-validation generator, default=None 5

The default cross-validation generator used is Stratified K-Folds.

If an integer is provided, it specifies the number of folds, ``n_folds``, used.

See the module `:mod:`sklearn.model_selection`` module for the list of possible cross-validation objects.

.. versionchanged:: 0.22

``cv`` default value if None changed from 3-fold to 5-fold.

scoring scoring: str or `'neg_log_loss'` callable, default=None

The scoring method to use for cross-validation. Options:

- str: see `:ref:`scoring_string_names`` for options.

- callable: a scorer callable object (e.g., function) with signature

``scorer(estimator, X, y)``.

See `:ref:`scoring_callable`` for details.

- ``None``: `:ref:`accuracy`` `<accuracy_score>` is used.

.. versionchanged:: 1.11

The default will change from None, i.e. accuracy, to 'neg_log_loss' in version 1.11.

solver solver: {'lbfgs', 'lib-linear', 'newton-cg', 'newton-cholesky', 'sag', 'saga'}, default='lbfgs'

Algorithm to use in the optimization problem. Default is 'lbfgs'.

To choose a solver, you might want to consider the following aspects:

- 'lbfgs' is a good default solver because it works reasonably well for a wide class of problems.
- For :term:`multiclass` problems (`n_classes >= 3`), all solvers except 'liblinear' minimize the full multinomial loss, 'liblinear' will raise an error.
- 'newton-cholesky' is a good choice for `n_samples >> n_features * n_classes`, especially with one-hot encoded categorical features with rare categories. Be aware that the memory usage of this solver has a quadratic dependency on `n_features * n_classes` because it explicitly computes the full Hessian matrix.

- For small datasets, 'liblinear' is a good choice, whereas 'sag' and 'saga' are faster for large ones;
- 'liblinear' might be slower in `:class:`LogisticRegressionCV`` because it does not handle warm-starting.
- 'liblinear' can only handle binary classification by default. To apply a one-versus-rest scheme for the multiclass setting one can wrap it with the `:class:`~sklearn.multiclass.OneVsRestClassifier``.

.. warning::

The choice of the algorithm depends on the penalty ('l1_ratio=0' for L2-penalty, 'l1_ratio=1' for L1-penalty and '0 < l1_ratio < 1' for Elastic-Net) chosen and on (multinomial) multiclass support:

```
=====
=====
solver l1_ratio multinomial
multiclass
=====
=====
'lbfgs' l1_ratio=0 yes
'liblinear' l1_ratio=1 or
l1_ratio=0 no
'newton-cg' l1_ratio=0 yes
'newton-cholesky' l1_ratio=0
yes
'sag' l1_ratio=0 yes
'saga' 0<=l1_ratio<=1 yes
```

=====

.. note::
'sag' and 'saga' fast convergence is only guaranteed on features with approximately the same scale. You can preprocess the data with a scaler from :mod:`sklearn.preprocessing`.

.. versionadded:: 0.17
Stochastic Average Gradient (SAG) descent solver. Multinomial support in version 0.18.

.. versionadded:: 0.19
SAGA solver.

.. versionadded:: 1.2
newton-cholesky solver. Multinomial support in version 1.6.

max_iter max_iter: int, default=100

Maximum number of iterations of the optimization algorithm.

random_state random_state: 42
int, RandomState instance, default=None

Used when `solver='sag'`, 'saga' or 'liblinear' to shuffle the data.

Note that this only applies to the solver and not the cross-validation generator. See :term:`Glos-

sary <random_state>` for details.

`use_legacy_attributes` True
`use_legacy_attributes:` bool,
default=True

If True, use legacy values for attributes:

- ``C_`` is an ndarray of shape (n_classes,) with the same value repeated
- ``l1_ratio_`` is an ndarray of shape (n_classes,) with the same value repeated
- ``coefs_paths_`` is a dict with class labels as keys and ndarrays as values
- ``scores_`` is a dict with class labels as keys and ndarrays as values
- ``n_iter_`` is an ndarray of shape (1, n_folds, n_cs) or similar

If False, use new values for attributes:

- ``C_`` is a float
- ``l1_ratio_`` is a float
- ``coefs_paths_`` is an ndarray of shape (n_folds, n_l1_ratios, n_cs, n_classes, n_features)
For binary problems (n_classes=2), the 2nd last dimension is 1.
- ``scores_`` is an ndarray of shape (n_folds, n_l1_ratios, n_cs)
- ``n_iter_`` is an ndarray of

shape (n_folds, n_l1_ratios,
n_cs)

.. versionchanged:: 1.10

The default will change from
True to False in version 1.10.

.. deprecated:: 1.10

`use\_legacy\_attributes` will
be deprecated in version 1.10
and be removed in
1.12.

Cs Cs: int or list of floats, de- 10
fault=10

Each of the values in Cs de-
scribes the inverse of regular-
ization

strength. If Cs is as an int,
then a grid of Cs values are
chosen

in a logarithmic scale be-
tween 1e-4 and 1e4.

Like in support vector ma-
chines, smaller values specify
stronger
regularization.

fit_intercept fit_intercept: True
bool, default=True

Specifies if a constant (a.k.a.
bias or intercept) should be
added to the decision func-
tion.

dual dual: bool, default=False False

Dual (constrained) or primal
(regularized, see also
:ref:`this equation <regular-
ized-logistic-loss>`) formula-
tion. Dual formulation
is only implemented for l2

penalty with liblinear solver.
Prefer dual=False when
n_samples > n_features.

penalty: {'l1', 'l2', 'elasticnet'}, default='l2'
'deprecated'

Specify the norm of the
penalty:

- 'l2': add an L2 penalty term
(used by default);
- 'l1': add an L1 penalty
term;
- 'elasticnet': both L1 and L2
penalty terms are added.

.. warning::

Some penalties may not work
with some solvers. See the pa-
rameter

'solver' below, to know the
compatibility between the
penalty and
solver.

.. deprecated:: 1.8

'penalty' was deprecated in
version 1.8 and will be re-
moved in 1.10.

Use 'l1_ratio' and 'C'
instead. 'l1_ratio=0' for
'penalty='l2',

'l1_ratio=1'

for 'penalty='l1', 'l1_ratio'
set to any float between 0 and
1

for 'penalty='elasticnet',
and 'C=np.inf' for
'penalty=None'.

tol: float, default=1e-4 0.0001

Tolerance for stopping criteria.

`class_weight` `class_weight`: None
dict or 'balanced', default=None

Weights associated with classes in the form `{class_label: weight}`.
If not given, all classes are supposed to have weight one.

The "balanced" mode uses the values of `y` to automatically adjust weights inversely proportional to class frequencies in the input data as `n_samples / (n_classes * np.bincount(y))`.

Note that these weights will be multiplied with `sample_weight` (passed through the fit method) if `sample_weight` is specified.

.. versionadded:: 0.17

`class_weight == 'balanced'`

`n_jobs` `n_jobs`: int, default=None

Number of CPU cores used during the cross-validation loop.

`None` means 1 unless in a `:obj: joblib.parallel_backend` context.`

`-1` means using all processors. See `:term: Glossary <n_jobs>` for more details.

verbose verbose: int, de- 0
fault=0

For the 'liblinear', 'sag' and
'lbfgs' solvers set verbose to
any
positive number for ver-
bosity.

refit refit: bool, default=True True

If set to True, the scores are
averaged across all folds, and
the
coefs and the C that corre-
sponds to the best score is
taken, and a
final refit is done using these
parameters.
Otherwise the coefs, inter-
cepts and C that correspond
to the
best scores across folds are
averaged.

intercept_scaling 1.0
intercept_scaling: float, de-
fault=1

Useful only when the solver
'liblinear' is used
and 'self.fit_intercept' is set
to 'True'. In this case, 'x' be-
comes
'[x, self.intercept_scaling]',
i.e. a "synthetic" feature with
constant value equal to
'intercept_scaling' is ap-
pended to the instance vector.
The intercept becomes
'intercept_scaling *
synthetic_feature_weight'.

.. note::

The synthetic feature weight is subject to L1 or L2 regularization as all other features.

To lessen the effect of regularization on synthetic feature weight (and therefore on the intercept) ``intercept_scaling`` has to be increased.

Name	Type	Value
<code>C_</code> <code>C_</code> : ndarray of shape (n_classes,) or (1,)	ndarray[float64](1,)	[0.36]
The value of C that maps to the best score, repeated n_classes times. If refit is set to False, the best C is the average of the C's that correspond to the best score for each fold. <code>`C_`</code> is of shape (1,) when the problem is binary. See also parameter <code>`use_legacy_attributes`</code>.		
<code>Cs_</code> <code>Cs_</code> : ndarray of shape (n_cs)	ndarray[float64](10,)	[0. , 0. , 0.01,..., 166.81, 1291.55,10000.]
Array of C i.e. inverse of regularization parameter values used for cross-validation.		
<code>classes_</code> <code>classes_</code> : ndarray of shape (n_classes,)	ndarray[float64](2,)	[0.,1.]
A list of class labels known to the classifier.		

`coef_`: ndarray of shape `(1, n_features)` or `(n_classes, n_features)`

`[[ 0.55, 0.26, -0.14, ..., -0.32, 0.46, -0.24]]`

Coefficient of the features in the decision function.

``coef_`` is of shape `(1, n_features)` when the given problem is binary.

`coefs_paths_`: dict of ndarray of shape `(n_folds, n_cs, n_dof)` or `(n_folds, n_cs, n_l1_ratios, n_dof)`

`{np.float64(1.0): array([[[[ 0.... 5, 10, 4, 45]])]}`

A dict with classes as the keys, and the path of coefficients obtained during cross-validating across each fold (``n_folds``) and then across each Cs (``n_cs``).

The size of the coefficients is the number of degrees of freedom (``n_dof``), i.e. without intercept ``n_dof=n_features`` and with intercept ``n_dof=n_features+1``.

If ``penalty='elasticnet``, there is an additional dimension for the number of

`l1_ratio` values (``n_l1_ratios``), which gives a shape of

```(n_folds, n_cs, n_l1_ratios_, n_dof)```.

See also parameter ``use_legacy_attributes``.

`intercept_` `intercept_`: ndarray of shape (1,) or (n\_classes,) ndarray[float64](1,) [-2.59]

Intercept (a.k.a. bias) added to the decision function.

If `'fit_intercept'` is set to False, the intercept is set to zero.

`'intercept_'` is of shape (1,) when the problem is binary.

`l1_ratio` `l1_ratio`: ndarray of shape (n\_classes,) or (n\_classes - 1,) ndarray[float64](1,) [0.7]

The value of `l1_ratio` that maps to the best score, repeated `n_classes` times.

If `refit` is set to False, the best `l1_ratio` is the average of the `l1_ratio`'s that correspond to the best score for each fold.

`'l1_ratio_'` is of shape (1,) when the problem is binary.

See also parameter `'use_legacy_attributes'`.

`l1_ratios` `l1_ratios`: ndarray of shape (n\_l1\_ratios) ndarray[float64](4,) [0.1,0.5,0.7,0.9]

Array of `l1_ratios` used for cross-validation. If `l1_ratios=None` is used (i.e. penalty is not 'elasticnet'), this is set to ```[None]```.

`n_features_in_` int 44  
`n_features_in_`: int

Number of features seen during :term:`fit`.

.. versionadded:: 0.24

`n_iter_`: ndarray of shape (1, `n_folds`, `n_cs`) or (1, `n_folds`, `n_cs`, `n_l1_ratios`)

Actual number of iterations for all classes, folds and Cs.

If `penalty='elasticnet'`, the shape is `(1, n_folds, n_cs, n_l1_ratios)`.

See also parameter `'use_legacy_attributes'`.

`scores_`: dict

A dict with classes as the keys, and the values as the grid of scores obtained during cross-validating each fold.

The same score is repeated across all classes. Each dict value

has shape `((n_folds, n_cs))` or `((n_folds, n_cs, n_l1_ratios))` if

`penalty='elasticnet'`.

See also parameter `'use_legacy_attributes'`.

```
[[[[1, 1, 1, 1], [3, 1, 1, 1], [13, 5, 1, 1], ..., [963, 983, 998, 1001], [542, 464, 411, 353], [109, 84, 70, 55]], [[1, 1, 1, 1], [3, 1, 1, 1], [12, 5, 5, 1], ..., [119, 127, 133, 136], [37, 41, 41, 43], [5, 5, 7, 7]], [[1, 1, 1, 1], [3, 1, 1, 1], [12, 3, 1, 1], ..., [96, 100, 102, 104], [27, 30, 30, 32], [7, 5, 7, 5]], [[1, 1, 1, 1], [3, 1, 1, 1], [13, 3, 1, 1], ..., [91, 98, 102, 105], [26, 30, 32, 33], [5, 5, 5, 6]], [[1, 1, 1, 1], [3, 1, 1, 1], [13, 3, 3, 3], ..., [98, 104, 105, 107], [30, 31, 34, 34], [5, 5, 5, 5]]]]
```

```
{np.float64(1.0): array([[[[-0.4...0.29902366]]]])}
```

Analysis goes here:

## Compare Models

```
def find_best_cutoff(y_true, y_probs):
 cutoffs = np.linspace(0.1, 0.9, 81)
 best_cutoff = 0.5
 best_acc = 0
```

```

 for c in cutoffs:
 acc = accuracy_score(y_true, (y_probs >= c).astype(int))
 if acc > best_acc:
 best_acc = acc
 best_cutoff = c
 return best_cutoff

models = {'Ridge': ridge_cv, 'LASSO': lasso_cv, 'Elastic Net': elastic_cv}
results = {}

for name, model in models.items():
 # Get predicted probabilities
 train_probs = model.predict_proba(X_train_scaled)[: , 1]
 test_probs = model.predict_proba(X_test_scaled)[: , 1]

 # Tune cutoff on train data
 best_c = find_best_cutoff(y_train, train_probs)

 # Calculate accuracy on test data using the tuned cutoff
 test_preds = (test_probs >= best_c).astype(int)
 test_acc = accuracy_score(y_test, test_preds)

 results[name] = {'Tuned Cutoff': best_c, 'Test Accuracy': test_acc}

Display Comparison Table
print(pd.DataFrame(results).T)

```

	Tuned Cutoff	Test Accuracy
Ridge	0.46	0.886621
LASSO	0.44	0.888889
Elastic Net	0.45	0.888889

Analysis goes here:

### Bootstrap Confidence Intervals

```

from sklearn.utils import resample
from sklearn.linear_model import LogisticRegression

Assuming 'LASSO' or 'Elastic Net' performed best; replace `best_model_class`
accordingly
n_iterations = 1000
boot_coefs = []

```

```

Filter to your selected optimal model parameter setting for bootstrapping
best_C = lasso_cv.C_[0]

for i in range(n_iterations):
 # Resample training data
 X_boot, y_boot = resample(X_train_scaled, y_train, random_state=i)

 # Refit model with the fixed optimal hyperparameter
 boot_model = LogisticRegression(l1_ratio=0, solver='saga', C=best_C,
max_iter=5000)
 boot_model.fit(X_boot, y_boot)
 boot_coefs.append(boot_model.coef_.flatten())

boot_coefs = np.array(boot_coefs)

Calculate 95% Percentile Confidence Intervals
lower_bounds = np.percentile(boot_coefs, 2.5, axis=0)
upper_bounds = np.percentile(boot_coefs, 97.5, axis=0)

ci_df = pd.DataFrame({
 'Feature': X_train_clean.columns,
 'Optimal_Coef': lasso_cv.coef_.flatten(),
 '2.5%': lower_bounds,
 '97.5%': upper_bounds
})
print(ci_df)

```

	Feature	Optimal_Coef	2.5%	97.5%
0	C(BusinessTravel)[T.Travel_Frequently]	0.513353	0.367883	0.934546
1	C(BusinessTravel)[T.Travel_Rarely]	0.226238	0.044304	0.663047
2	C(EducationField)[T.Life_Sciences]	-0.119954	-0.611629	0.021477
3	C(EducationField)[T.Marketing]	0.000000	-0.345682	0.174595
4	C(EducationField)[T.Medical]	-0.161846	-0.620700	-0.016774
5	C(EducationField)[T.Other]	-0.173115	-0.612673	-0.009208
6	C(EducationField)[T.Technical_Degree]	0.008071	-0.315408	0.212061
7	C(Education)[T.2]	0.000000	-0.279433	0.291227
8	C(Education)[T.3]	0.053063	-0.198085	0.398597
9	C(Education)[T.4]	0.000000	-0.276931	0.308472
10	C(Education)[T.5]	0.000000	-0.464194	0.258851
11	C(Gender)[T.Male]	0.119498	-0.045941	0.394432
12	C(JobLevel)[T.2]	-0.539264	-1.017578	-0.286225

13	C(JobLevel)[T.3]	0.000000	-0.310667	0.315806
14	C(JobLevel)[T.4]	-0.072301	-0.565537	0.254005
15	C(JobLevel)[T.5]	0.051034	-0.295496	0.533554
16	C(JobRole)[T.Human Resources]	0.176973	0.013816	0.392438
17	C(JobRole)[T.Laboratory Technician]	0.338533	0.099908	0.699553
18	C(JobRole)[T.Manager]	0.000000	-0.582317	0.243292
19	C(JobRole)[T.Manufacturing Director]	0.000000	-0.347873	0.412102
20	C(JobRole)[T.Research Director]	-0.292788	-0.842207	-0.089275
21	C(JobRole)[T.Research Scientist]	-0.111491	-0.476750	0.202117
22	C(JobRole)[T.Sales Executive]	0.462553	0.225280	0.898356
23	C(JobRole)[T.Sales Representative]	0.284015	0.096687	0.578364
24	C(MaritalStatus)[T.Married]	0.055648	-0.112015	0.407936
25	C(MaritalStatus)[T.Single]	0.542564	0.431350	0.927917
26	C(OverTime)[T.Yes]	0.810269	0.685096	1.134513
27	Age	-0.365524	-0.751043	-0.073900
28	DistanceFromHome	0.313374	0.177999	0.578817
29	EnvironmentSatisfaction	-0.422895	-0.724182	-0.269502
30	HourlyRate	-0.048943	-0.279947	0.166252
31	JobInvolvement	-0.338418	-0.601872	-0.185766
32	JobSatisfaction	-0.361156	-0.636716	-0.218541
33	MonthlyRate	0.112511	-0.048997	0.372158
34	NumCompaniesWorked	0.451654	0.320157	0.752756
35	PercentSalaryHike	-0.030742	-0.292088	0.154431
36	RelationshipSatisfaction	-0.315474	-0.590433	-0.178688
37	TotalWorkingYears	-0.385696	-0.904970	0.014742
38	TrainingTimesLastYear	-0.134508	-0.398862	0.045242
39	WorkLifeBalance	-0.240868	-0.518599	-0.059003
40	YearsAtCompany	0.000000	-0.398301	0.493669
41	YearsInCurrentRole	-0.305697	-0.765551	-0.076390
42	YearsSinceLastPromotion	0.425366	0.249491	0.865491
43	YearsWithCurrManager	-0.236055	-0.664991	0.126395

Analysis goes here:

### Model Comparison using Bayesian Logistic Regression

```
import bambi as bmb
import arviz as az

Initialize a Bayesian model with a regularizing Normal prior (Ridge equivalent)
bayes_model = bmb.Model("Left ~ Age + DistanceFromHome + C(OverTime)",
data=train_df, family="bernoulli")
Fit using MCMC sampling
results_bayes = bayes_model.fit(draws=1000, tune=1000, random_seed=42)
```

```
print(az.summary(results_bayes))
```

```
az.plot_trace(results_bayes)
```

```
Modeling the probability that Left==1
Initializing NUTS using jitter+adapt_diag...
Multiprocess sampling (4 chains in 4 jobs)
NUTS: [Intercept, Age, DistanceFromHome, C(OverTime)]
```

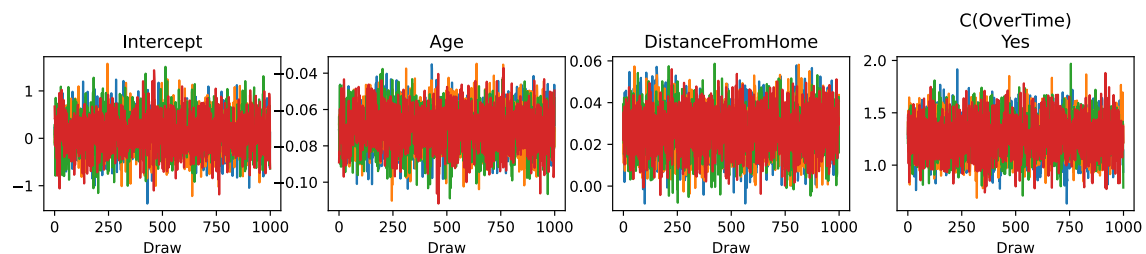
```
Output()
```

Sampling 4 chains for 1\_000 tune and 1\_000 draw iterations (4\_000 + 4\_000 draws total) took 3 seconds.

	mean	sd	eti89_lb	eti89_ub	ess_bulk	ess_tail	r_hat	\
Intercept	0.1	0.388	-0.5	0.71	4878	3048	1.00	
Age	-0.0705	0.0109	-0.088	-0.053	3874	3127	1.00	
DistanceFromHome	0.0261	0.0107	0.0088	0.044	4062	3003	1.00	
C(OverTime)[Yes]	1.27	0.181	0.98	1.6	4367	3319	1.00	

	mcse_mean	mcse_sd
Intercept	0.0056	0.004
Age	0.00018	0.00013
DistanceFromHome	0.00017	0.00011
C(OverTime)[Yes]	0.0027	0.0019



Analysis goes here:

### Business Implications

The model suggests:

1. Vehicle age is the strongest continuous driver of depreciation.
2. Mileage has a significant but smaller effect.
3. Vehicle model contributes substantial value differences.
4. Trucks generally retain value better than passenger sedans.

### **Recommendations**

Organizations purchasing used vehicles should:

- Prioritize lower-age vehicles when possible.
- Consider mileage alongside age rather than in isolation.
- Use model-specific adjustments when evaluating value.
- Compare observed asking prices against model predictions to identify unusually favorable purchasing opportunities.

### **Future Work**

Future improvements could include:

1. Incorporating original MSRP.
2. Including trim-level information.
3. Adding accident-history variables.
4. Incorporating local economic indicators.

### **Conclusion**

The final regression model explains approximately 82% of the variation in used vehicle selling prices and provides a practical framework for vehicle valuation. Vehicle age, mileage, model, and geographic location all contribute significantly to price determination. The resulting model can support data-driven purchasing and pricing decisions while highlighting areas for future model enhancement.